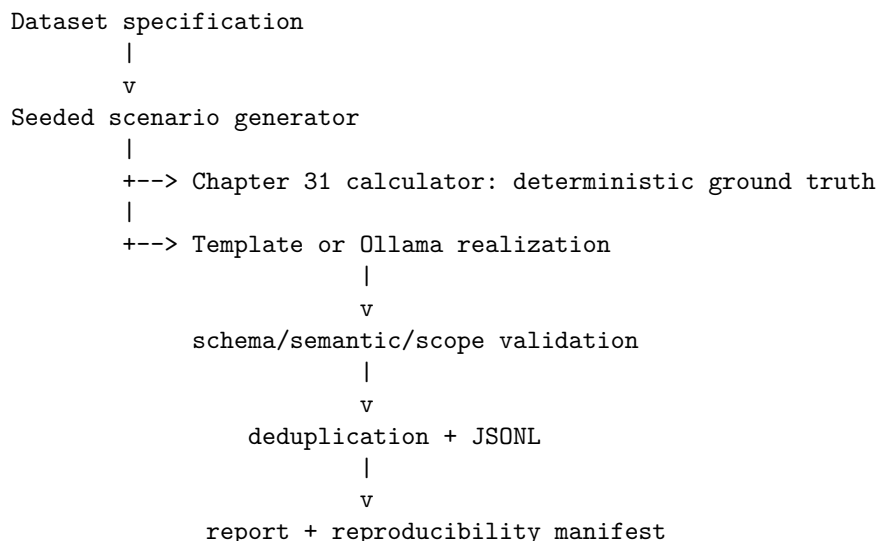


Contents

Chapter 32 — Synthetic Dataset Generator	1
Learning objectives	2
Scope	2
Setup	2
Quick start	3
Specification format	3
Command-line interface	4
generate	4
preview	5
validate	5
stats and inspect	5
reproduce	5
Exit codes and diagnostics	5
Generated record shape	6
Distributions and constraints	6
Architecture and module map	6
Chapter 31 calculator boundary	7
Optional Ollama realization	8
Optional desktop UI	8
Reports and reproducibility	9
Validation and failure behavior	9
Testing	10
Development workflow	10
Safety and scope disclaimer	10

Chapter 32 — Synthetic Dataset Generator

A deterministic-first synthetic dataset generator for evaluation data. The application demonstrates a strict scenario/ground-truth/language boundary:



The language generator is never the authority for ground truth. It receives a structured scenario and produces wording; the calculator, validator, and artifact writer own correctness and publication decisions.

Learning objectives

Chapter 32 demonstrates how to:

- Generate evaluation cases from a declarative YAML/JSON specification rather than an unconstrained prompt.
- Separate abstract scenario generation from natural-language realization.
- Reuse Chapter 31’s deterministic mortgage calculator through a calculator protocol and registry.
- Keep all deterministic behavior on a run-local seeded `random.Random` instance.
- Implement bounded numeric, categorical, and weighted distributions.
- Evaluate constraint expressions with a safe grammar instead of unrestricted `eval`.
- Produce controlled template variation while preserving intent and input values.
- Treat optional Ollama output as untrusted language that must pass semantic validation.
- Detect exact and configurable near duplicates before publication.
- Write stable JSONL, quality reports, and manifests for reproducibility and auditability.
- Use bounded retries, structured failures, atomic artifact publication, and explicit partial-run semantics.
- Expose one service through both a scriptable CLI and an optional PyQt5 boundary.

Scope

The generator supports:

- Declarative YAML and JSON specifications.
- Typed fields: `string`, `decimal`, `integer`, and finite `enum`.
- Weighted categories and `uniform`, `lognormal`, `choice`, and `values` distributions.
- Safe arithmetic/comparison constraints over scenario fields.
- Valid, boundary, invalid, ambiguous, unsupported, and underspecified categories when declared by a specification.
- Deterministic mortgage-question templates for payment, principal, term, and rate intents.
- Optional local Ollama wording through `/api/generate`.
- Schema, semantic-equivalence, declared-scope, and exact-deduplication gates.
- JSONL records shaped like Chapter 31 evaluation cases: `case_id`, `category`, `question`, nested `expected`, and `metadata`.
- Quality reports, SHA-256 hashes, reproducibility manifests, preview mode, and partial-run reporting.
- CLI commands for generation, validation, statistics, inspection, and reproduction.
- An optional PyQt5 GUI entry point that delegates to the same service boundary.

The generator does **not** collect web-scale data, discover domains autonomously, train a model, use an LLM to calculate truth, require hosted embeddings for semantic deduplication, or provide production annotation workflows. The bundled domain adapter is the Chapter 31 fixed-rate mortgage calculator; other domains must be registered explicitly.

Setup

Python 3.12 and `uv` are required.

```
cd labs/week4/chapter32
uv sync --extra test
uv sync --extra gui # optional PyQt5 GUI dependencies
```

The Chapter 31 calculator is installed as a local path dependency. The test and template paths do not require:

- An API key.
- A running Ollama daemon.
- Network access.
- A visible GUI display.

Quick start

Generate a deterministic 30-record dataset using the bundled mortgage specification:

```
uv run synthgen generate \  
  examples/mortgage.yaml \  
  --size 30 \  
  --seed 42
```

The specification defaults write:

```
evals/generated.jsonl  
evals/report.json  
evals/manifest.json
```

Inspect the generated artifacts:

```
uv run synthgen validate evals/generated.jsonl  
uv run synthgen stats evals/generated.jsonl  
uv run synthgen inspect evals/generated.jsonl  
uv run synthgen reproduce evals/manifest.json
```

A successful deterministic run prints a summary similar to:

Generated 30/30 records

`reproduce` returns exit code 0 when the referenced dataset hash and normalized records match a fresh deterministic run.

Run the tests:

```
uv run pytest -q
```

Specification format

A specification has three required sections: **dataset**, **schema**, and **categories**. Constraints and realization settings are optional.

The bundled `examples/mortgage.yaml` begins with:

```
dataset:  
  name: mortgage_questions  
  domain: mortgage  
  size: 30  
  seed: 42  
  max_attempts: 300  
  output: evals/generated.jsonl  
  report: evals/report.json  
  manifest: evals/manifest.json  
  
schema:  
  fields:  
    - name: question  
      type: string  
      required: true  
      nullable: false  
    - name: intent  
      type: enum  
      values: [payment, principal, term, rate]  
      required: true
```

```

    nullable: false
  - name: principal
    type: decimal
    required: false
    nullable: true
    minimum: 1

categories:
  payment:
    weight: 0.25
    principal: {distribution: values, values: [100000, 250000, 500000]}
    annual_rate: {distribution: values, values: [3.5, 5.0, 6.5]}
    term_years: {distribution: values, values: [15, 30]}

realization:
  method: template
  max_regenerations: 3

```

Every field descriptor defines **name**, **type**, **required**, and **nullable**. Numeric fields may specify **minimum**, **maximum**, and **distribution**; enum fields require finite **values**. Category weights are positive and must sum to 1.0 within 1e-9. `dataset.domain` selects a registered calculator.

Command-line interface

generate

Validate a specification, sample scenarios, calculate truth, realize language, validate candidates, deduplicate, and publish JSONL/report/manifest artifacts:

```

uv run synthgen generate examples/mortgage.yaml \
  --size 1000 \
  --seed 42 \
  --output evals/generated.jsonl \
  --report evals/report.json \
  --manifest evals/manifest.json

```

Options:

Option	Meaning
<code>--size N</code>	Override the requested accepted-record count.
<code>--seed N</code>	Override the specification seed.
<code>--output PATH</code>	Override the JSONL path.
<code>--report PATH</code>	Override the report path.
<code>--manifest PATH</code>	Override the manifest path.
<code>--method template\ ollama</code>	Select deterministic templates or Ollama wording.
<code>--model MODEL</code>	Ollama model name.
<code>--host URL</code>	Ollama endpoint.
<code>--max-attempts N</code>	Bound candidate attempts. The default is <code>max(size * 5, size + 10)</code> .
<code>--allow-partial</code>	Publish accepted records with <code>complete: false</code> after exhaustion.
<code>--include-raw</code>	Permit bounded raw model diagnostics when explicitly requested.
<code>--verbose [INFO\ DEBUG]</code>	Emit metadata, or metadata plus raw model payload diagnostics.

Without `--allow-partial`, an exhausted run publishes no artifacts and returns exit code 1. The template path is offline and deterministic; identical specification, seed, generator version, adapter, and options produce identical normalized output.

preview

Generate at most 100 records without writing production artifacts:

```
uv run synthgen preview examples/mortgage.yaml --size 5 --seed 42
```

Preview records are printed as JSON objects. `--size 101` is rejected as a usage error.

validate

Validate the basic JSONL record contract without generating new records:

```
uv run synthgen validate evals/generated.jsonl
```

The command checks non-empty input, required C-05 fields, nested expected outcomes, and metadata. It prints invalid case IDs and returns 1 when records fail validation.

stats and inspect

Both commands are read-only:

```
uv run synthgen stats evals/generated.jsonl
uv run synthgen inspect evals/generated.jsonl
```

`stats` prints record count, category counts, and realization methods. `inspect` prints a bounded sample of records and metadata for auditing.

reproduce

Re-run the manifest configuration and compare its dataset hash and normalized deterministic records:

```
uv run synthgen reproduce evals/manifest.json
```

Manifest-relative dataset/report/manifest paths are resolved relative to the manifest directory. Unsupported artifact versions fail unless `--force` is supplied.

Exit codes and diagnostics

Exit	Meaning
0	Successful generation, validation, statistics, inspection, preview, or reproduction.
1	Attempt budget exhausted, invalid records, or reproduction mismatch.
2	Usage error, malformed input, invalid specification, or unsupported artifact version.
5	Ollama/model failure or artifact write failure.

Diagnostics use Loguru and are written to `stderr`; normal dataset and command output remains separate:

- No verbosity flag: quiet.
- `--verbose` or `--verbose INFO`: metadata only.
- `--verbose DEBUG`: metadata plus bounded raw Ollama prompt/response diagnostics.

Diagnostics do not affect generated records or deterministic hashes.

Generated record shape

Each accepted JSONL line follows the Chapter 31 evaluation-case boundary:

```
{
  "case_id": "payment-000000",
  "category": "payment",
  "question": "What is the monthly payment on a $250,000.00 mortgage at 5.0% for 30 years?",
  "expected": {
    "intent": "payment",
    "outcome": "calculated",
    "fields": {
      "payments": "360",
      "payment": "1342.05"
    }
  },
  "metadata": {
    "scenario_id": "payment-000000",
    "generator": "template",
    "template_id": "payment_01",
    "model": null,
    "seed": 123456,
    "spec_hash": "sha256:..."
  }
}
```

expected is deterministic calculator output. It is never copied from an LLM response. Decimal values are serialized as strings to preserve precision. Rejected candidates never consume an accepted record ID and never appear in the JSONL.

Distributions and constraints

The supported distribution contracts are:

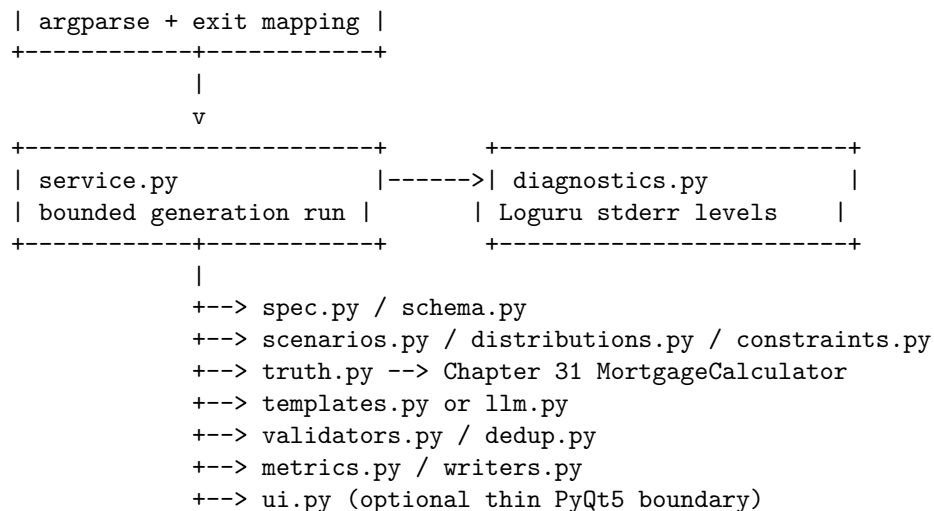
Distribution	Semantics
uniform	Sample from <code>[min, max)</code> using the run-local RNG.
lognormal	Sample <code>exp(rng.normalvariate(mu, sigma))</code> , then apply declared finite bounds.
choice	Select from finite values , optionally using positive weights .
values	Select one value from a finite ordered list.

Constraint expressions use a safe restricted grammar. Supported operations are identifiers, literals, `+`, `-`, `*`, `/`, parentheses, comparisons, **and**, **or**, and membership in finite lists. Calls, attribute access, indexing, imports, assignment, comprehensions, and unrestricted **eval** are rejected. Missing fields and division by zero become deterministic constraint failures.

All category selection and field sampling uses a run-local `random.Random(seed)`. Global random state and wall-clock time do not influence scenario IDs or candidate seeds.

Architecture and module map

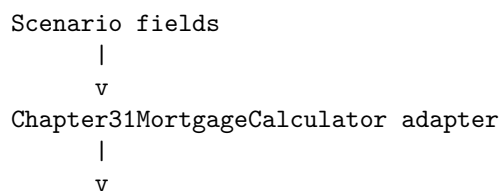
```
+-----+
| cli.py |
```



Module	Responsibility
<code>models.py</code>	Frozen scenarios, ground truth, realizations, validation, duplicate, and calculator protocol types.
<code>errors.py</code>	Stable structured specification, calculator, model, exhaustion, and artifact errors.
<code>spec.py</code>	YAML/JSON loading, typed descriptors, defaults, normalization, and specification hashes.
<code>schema.py</code>	Declarative schema boundary and record-shape definitions.
<code>distributions.py</code>	Seeded numeric, categorical, and weighted sampling.
<code>constraints.py</code>	Safe expression parsing and deterministic checks.
<code>scenarios.py</code>	Category selection, field sampling, stable scenario IDs, and constraint gates.
<code>truth.py</code>	Calculator registry and the Chapter 31 adapter; no language-model dependency.
<code>templates.py</code>	Offline controlled mortgage-question wording.
<code>llm.py</code>	Optional Ollama realization with bounded raw diagnostics.
<code>validators.py</code>	Schema, extraction, semantic-equivalence, and scope checks.
<code>dedup.py</code>	Normalized exact deduplication and optional near-duplicate classification.
<code>metrics.py</code>	Stable acceptance, rejection, category, duplicate, and method metrics.
<code>writers.py</code>	Sorted UTF-8 JSON/JSONL, hashes, staging, and atomic publication.
<code>service.py</code>	Bounded generation lifecycle, provenance, and reproduction.
<code>cli.py</code>	Scriptable command surface and exit-code mapping.
<code>ui.py</code>	Optional PyQt5 boundary delegating to the shared service.

Chapter 31 calculator boundary

Chapter 32 reuses Chapter 31 rather than duplicating mortgage arithmetic. `truth.py` is the sole integration boundary:



```

mortgage.CalculationRequest
|
v
Chapter 31 deterministic calculator
|
v
Generic GroundTruth

```

The adapter supports Chapter 31's four inverse calculations:

- Payment from principal, rate, and term.
- Principal from payment, rate, and term.
- Payment count from principal, rate, and payment.
- Rate from principal, payment, and payment count.

Payment-too-low cases are represented as the deterministic `payment_too_low` outcome. The adapter never receives an LLM client, and changing the realization method cannot change the calculator result.

A replacement calculator must implement:

```

class GroundTruthCalculator(Protocol):
    @property
    def version(self) -> str: ...

    def calculate(self, scenario: Scenario) -> GroundTruth: ...

```

Register it under the specification's `dataset.domain`; unknown domains fail before scenario generation.

Optional Ollama realization

Install and start Ollama separately, then pull a model:

```
ollama pull llama3.2
```

Generate using the real-model boundary:

```

uv run synthgen generate examples/mortgage.yaml \
  --size 30 \
  --seed 42 \
  --method ollama \
  --model llama3.2 \
  --host http://127.0.0.1:11434 \
  --verbose INFO

```

The adapter sends a fixed scenario and a wording instruction to `/api/generate`. It does not give the model authority over category, expected outcome, or calculator fields. Empty, malformed, unavailable, or failed model responses become structured model failures; retries are bounded by `max_regenerations`.

The default dataset/report omit raw prompts and responses. `--verbose DEBUG` logs bounded payloads to `stderr`; `--include-raw` permits bounded raw response excerpts where explicitly retained. Raw content is capped at 4,000 characters per candidate.

The deterministic/template path never opens network sockets. An Ollama failure returns exit code 5; it does not silently switch to a different truth source.

Optional desktop UI

Launch the optional PyQt5 entry point with:

```
uv run synthgen-gui
```

The GUI boundary is intentionally thin and shares the generator service rather than duplicating scenario, calculator, validation, or reporting logic. It provides the bounded preview surface and diagnostics selector (Off, INFO, DEBUG). When `ollama` realization is selected, the GUI shows model and host fields defaulting to `llama3.2` and `http://127.0.0.1:11434`; those values are passed to `OllamaRealizer` for Preview. Construction does not generate data. Use the CLI for scripted artifact paths and complete batch workflows.

For headless environments:

```
QT_QPA_PLATFORM=offscreen uv run pytest tests/test_ui.py -q
```

Reports and reproducibility

A report records:

- Requested, attempted, accepted, and rejected counts.
- `complete` status.
- Accepted category counts.
- Validation and rejection counts.
- Exact and near duplicate counts.
- Realization methods.
- Acceptance and rejection rates.
- Deterministic failure records with stage, reason, and scenario ID.
- The manifest reference.

Rates use six decimal places and serialize empty denominators as `null`. Category percentages use accepted records; validation and duplicate rates use attempted candidates. Unavailable generation cost is `null`, not zero.

A manifest records:

- `manifest_version` and `generator_version`.
- Specification and dataset SHA-256 hashes.
- Dataset, report, manifest, and specification paths.
- Seed and requested size.
- Attempt budget.
- Adapter, model, temperature, and creation timestamp.

Artifact writes first use a run-specific staging directory. Without `--allow-partial`, ordinary failure publishes none of the required artifacts. With `--allow-partial`, accepted JSONL records and report/manifest artifacts are published with `complete: false`; reproduction rejects a partial run as a complete match.

Reproducibility example:

```
uv run synthgen generate examples/mortgage.yaml --size 30 --seed 42
cp evals/generated.jsonl /tmp/first.jsonl
uv run synthgen reproduce evals/manifest.json
cmp /tmp/first.jsonl evals/generated.jsonl
```

The manifest timestamp is provenance only and is excluded from normalized deterministic comparisons.

Validation and failure behavior

The generator has explicit failure stages:

Stage	Examples
Specification	Missing sections, malformed YAML/JSON, invalid weights, unknown methods, unregistered calculator.
Scenario	Impossible constraints, unbounded distributions, invalid sampled fields.

Stage	Examples
Calculator	Domain rejection and stable Chapter 31 calculator errors.
Realization	Empty template or malformed/unavailable Ollama output.
Semantic	Changed principal, rate, term, payment, intent, units, or expected outcome.
Scope	Taxes, insurance, HOA, adjustable-rate concepts, lender advice, or other undeclared concepts.
Deduplication	Exact normalized-text duplicate or configured near duplicate.
Artifact	Staging, serialization, hash, or final-path write failure.

Every rejection is retained in the report with its stage, reason, source scenario, and bounded candidate details. No rejected candidate consumes an accepted sequence number.

Testing

Run the complete Chapter 32 suite:

```
uv run pytest -q
```

The tests cover:

- Frozen contract types and Decimal preservation.
- YAML loading, specification defaults, malformed specifications, and deterministic hashes.
- Seeded distributions and safe constraint evaluation.
- Scenario IDs, category sampling, and calculator substitution.
- Chapter 31 payment, principal, term, rate, and payment-too-low outcomes.
- Template semantic preservation and exact/near deduplication.
- Stable JSONL/report/manifest artifact generation.
- Relative manifest paths, dataset hashes, and deterministic reproduction.
- CLI generation, preview bounds, validation, stats, inspection, and exit behavior.
- Ollama error handling and bounded model diagnostics when a transport test is supplied.
- Optional GUI construction in offscreen environments.

The checked-in manual-evaluation artifact is generated with:

```
uv run synthgen generate examples/mortgage.yaml --size 30 --seed 42
uv run synthgen validate evals/generated.jsonl
uv run synthgen stats evals/generated.jsonl
uv run synthgen inspect evals/generated.jsonl
uv run synthgen reproduce evals/manifest.json
```

Development workflow

```
cd labs/week4/chapter32
uv sync --extra test --extra gui
uv run pytest -q
uv run synthgen preview examples/mortgage.yaml --size 5 --seed 42
```

When changing scenario fields, templates, validation, or artifact contracts, update the corresponding tests and the authoritative SPEC.md. Preserve the deterministic/probabilistic boundary: generate parameters, calculate truth, then generate language.

Safety and scope disclaimer

This is an educational evaluation-data generator. Generated mortgage cases are not lender-specific quotes, underwriting decisions, or financial advice. The Chapter 31 domain model intentionally excludes taxes,

insurance, HOA fees, lender fees, adjustable-rate products, and other unsupported housing concepts. Unsupported concepts must remain rejected rather than being invented by a language model.